Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Carcassonne – Python

Rapport préliminaire

Anthony BEAUVIEUX, Tony HUANG, Maxence SALIN, Clément SIMON, Loïc THIN

1 Présentation du jeu

1.1 Contexte

Carcassonne est un jeu de stratégie et de placement de tuiles créé par Klaus-Jürgen Wrede en 2000, conçu pour 2 à 6 joueurs. Il s'agit d'un jeu stochastique à information parfaite, c'est-à-dire un jeu où le hasard intervient dans le tirage des tuiles mais où l'ensemble des éléments présents sur le plateau est visible de tous les joueurs.



FIGURE 1 – Tuile de départ

La version considérée ici utilise les 72 tuiles du jeu de base, auxquelles s'ajoutent les 12 tuiles de l'extension *The River*. Parmi les 72 tuiles se trouve la tuile de départ (voir figure 1) qui est placée au centre du plateau avant que la partie commence. Cette tuile est la même pour chaque partie comportant une *City* en haut (couleur marron), une *Road* qui traverse de gauche à droite et un *Field* en bas. Chaque joueur dispose de 7 pions de sa couleur pour marquer des points au cours de la partie. Le but du jeu est que les joueurs construisent progressivement le plateau, à tour de rôle, en posant des tuiles de manière cohérente et en plaçant stratégiquement leurs pions pour accumuler des points. Le vainqueur est le joueur qui totalise le plus de points à la fin de la partie.

1.2 Règles

Les règles de **Carcassonne** utilisées sont celles de la version 2012[1].

1.2.1 Spécificités

Tuile

Chaque tuile possède quatre côtés ainsi qu'un centre. Chaque côté correspond à un type de terrain parmi *Road*, *City* ou *Field*. Le centre peut représenter une *City*, un *Field*, une *Banner* ou un *Monastery*.



FIGURE 2 – Exemple d'une tuile avec une *City*, une *Road* et un *Field*

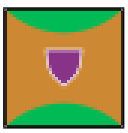


FIGURE 3 – Exemple d'une tuile où une *City* possède une *Banner*

Pion

Lorsqu'un pion est placé sur une tuile, il prend un rôle en fonction du type de terrain occupé : *Knight* sur une *City*, *Farmer* sur un *Field*, *Thief* sur une *Road* et *Monk* sur un *Monastery*.

1.2.2 Déroulement d'un tour

Les joueurs jouent à tour de rôle selon un ordre prédéfini (généralement dans le sens horaire, en commençant par le joueur le plus jeune). À son tour, un joueur effectue les actions suivantes dans l'ordre indiqué :

1. **Piocher une tuile** Le joueur pioche une tuile dans la pioche actuellement active. Cette tuile est visible par l'ensemble des joueurs.
2. **Placer la tuile** La tuile piochée doit être placée sur le plateau en respectant les règles suivantes :
   - La tuile doit être adjacente à au moins une autre tuile déjà posée.
   - Les côtés en contact doivent être de même type de terrain.
3. **Placer un pion (optionnel)** Le joueur peut placer un pion sur une partie de la tuile qu'il vient de poser (un des quatre côtés ou le centre).
4. **Restriction de placement des pions** Si une partie de terrain de la tuile posée est reliée à une partie du même type déjà occupée par un pion, aucun nouveau pion ne peut y être placé. Les *Road* et *City* agissent comme des séparateurs entre les terrains.
5. **Résolution des terrains complétés** Si la pose de la tuile complète un ou plusieurs terrains, ceux-ci sont résolus immédiatement et le joueur marque des points selon certaines règles. (voir partie 1.2.3)
6. **Retour des pions** Tous les pions associés à des *City*, *Road* ou *Monastery* complétés durant ce tour sont retirés du plateau et rendus à leurs propriétaires.

1.2.3 Score

Le score est calculé à deux moments différents : lors de la complétion d'un terrain (immédiatement après la pose d'une tuile) et à la fin de la partie (décompte final). Les joueurs ajoutent alors à leur score les points correspondant aux terrains selon les règles suivantes :

Road
Une route rapporte 1 point par tuile la composant au joueur possédant le plus de pions (*Thief*) sur cette route. En cas d'égalité, tous les joueurs concernés marquent ces points.

City
Une ville complète rapport 2 points par tuile la composant ainsi que 2 points par *Banner* présent, pour le joueur possédant le plus de pions (*Knight*). Les points rapportés deviennent 1 en cas de ville incomplète. En cas d'égalité, les points sont attribués à tous les joueurs concernés.

Monastery
Un monastère incomplet rapporte 1 point par tuile adjacente, y compris la tuile du monastère et les tuiles diagonales.

Field
Chaque *Farm* (ensemble de *Field* connectés ; un *Field* isolé constitue une *Farm*) est attribuée au joueur possédant le plus de pions (*Farmer*). Ce joueur marque 3 points par *City* complète adjacente à la *Farm*. En cas d'égalité, les points sont attribués à tous les joueurs concernés.

Le joueur ayant obtenu le plus grand nombre de points à l'issue du décompte final remporte la partie.

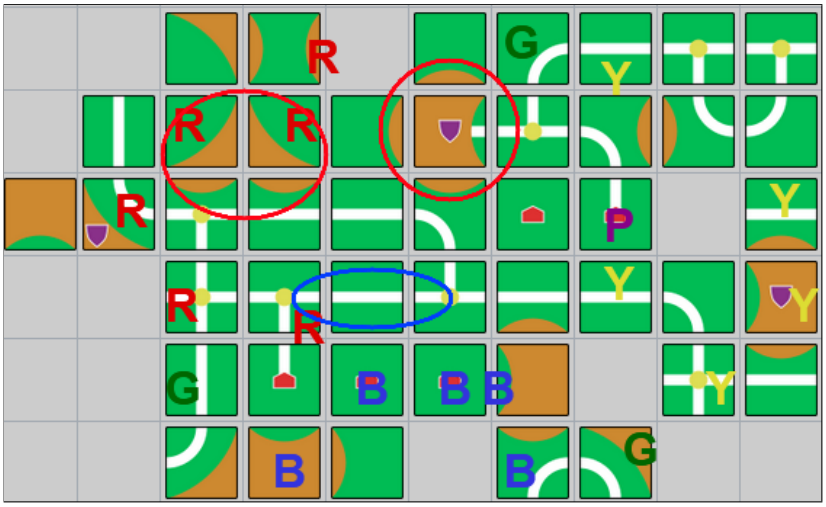


FIGURE 4 – Exemple de terrains complétés, avec deux villes entourées en rouge et une route entourée en bleu

1.3 Complexité

Carcassonne est un jeu aux règles simples, mais qui offre une grande richesse stratégique, les choix des joueurs se complexifiant à mesure que le plateau s'étend. Ainsi, sa complexité est comparable à celle de jeux classiques comme les échecs ou le shogi.

La complexité combinatoire du jeu est particulièrement élevée en raison du grand nombre de tuiles et des multiples façons dont elles peuvent être placées sur le plateau, ainsi que des stratégies variées que les joueurs peuvent adopter selon la configuration des tuiles et la position des pions. La complexité de l'arbre de jeu est estimée à environ 10^{194} [2], ce qui rend toute résolution complète du jeu par des méthodes de recherche exhaustive irréaliste. Par conséquent, la création d'une intelligence artificielle capable de jouer parfaitement à **Carcassonne** devient impossible. Le brute-force ne sera alors pas une solution envisageable pour notre projet.

2 Algorithmes et structures de données

2.1 Besoin F8

L'algorithme retenu repose sur la combinaison de deux sources d'information indépendantes :

- l'heure système en milliseconde.
- l'identifiant du processus courant.

```
fonction generate_seed():
  si une graine est fournie par l'utilisateur alors
      retourner la graine fournie
  sinon
      t ← heure système en milliseconde
      p ← identifiant du processus courant
      retourner t × p
  fin si
  fin fonction
```

Exemple : Temps en milliseconde(t) = 4118 pid(p) = 23259 seed = 4118 * 23259 = 95780562

3 Architecture

Le projet adopte une architecture 3-tiers visant à séparer clairement les responsabilités afin d'améliorer la maintenabilité, la testabilité et l'évolutivité du code. Cette architecture nous paraît pertinente compte tenu de l'aspect objet du langage python. De plus, le projet amène naturellement à une structuration de ce genre. Cette architecture est structurée autour de trois modules principaux : Présentation, Logique Métier et Données, organisés selon des dépendances unidirectionnelles.

3.1 Découpage en modules

3.1.1 Module Présentation

Le module Présentation est responsable des interactions avec l'utilisateur et de l'affichage de l'état du jeu. Il comprend notamment : la gestion des commandes à travers le shell en CLI ou des actions à partir de la GUI,

l'affichage du plateau, des tuiles, des meeples et des scores à travers CLI ou GUI,

Ce module ne contient aucune règle métier. Il délègue toute décision concernant la validité des actions ou l'évolution du jeu au module Logique Métier. Responsabilités principales :

— Interprétation des actions utilisateur ;

— Appels aux interfaces de la logique métier ;

— Rafraîchissement de l'affichage.

3.1.2 Module Logique Métier

Le module Logique Métier constitue le cœur du jeu. Il encapsule l'ensemble des règles du jeu et gère l'état interne de la partie. Il est notamment responsable : de la validation des coups (placement des tuiles, des meeples) à travers les algorithmes demandés,

— de la mise à jour du plateau (à travers la liste d'adjacence),
— du calcul des scores,
— de la gestion des tours de jeu et des conditions de fin de partie.
— des actions des IA (avec les fonctions d'évaluations)
— de l'état du serveur, avec gestion des parties

Ce module est totalement indépendant de l'interface graphique et de la persistance. Responsabilités principales :

— Implémentation des règles du jeu ;
— Gestion de l'état de la partie ;
— Gestion des IA de jeu ;
— Gestion des actions liées au serveur

3.1.3 Module Données

Le module Données est dédié à la persistance et à la gestion des données du jeu. Il permet notamment :

— la sauvegarde et le chargement d'une partie,
— la sauvegarde de historique,
— la sauvegarde des données relatives aux IA (structures de données) et des meilleurs coups enregistrés pour l'iterative deepening,
— la sauvegarde des données relatives aux parties en cours, à l'état du serveur et des joueurs.

Ce module n'implémente aucune règle métier et n'interagit jamais directement avec l'interface utilisateur. Responsabilités principales :

— Sauvegarde des informations liées au jeu de près ou de loin (avec IA et serveurs) ;
— Possibilité de sauvegarder les parties en cours pour les recharger (dans des fichiers au format répondant aux fonctionnalités).

3.2 Dépendances entre modules

Les dépendances entre modules sont strictement unidirectionnelles, conformément au principe d'architecture N-tiers :

— le module Présentation dépend du module Logique Métier ;
— le module Logique Métier dépend du module Données ;
— le module Données ne dépend d'aucun autre module.

Aucune dépendance directe n'existe entre la Présentation et les Données. Cette organisation garantit une séparation claire des responsabilités et limite le couplage entre les composants.

3.3 Interfaces principales

Les interactions entre modules sont réalisées exclusivement via des interfaces bien définies.
Interface Présentation → Logique Métier Exemples de services exposés :

```
place_tile(tile, position)
place_meeple(position)
get_game_state()
is_game_over()
```

Ces interfaces permettent à la présentation d'agir sur le jeu sans connaître l'implémentation interne des règles.
Interface Logique Métier → Données Exemples de services exposés :

```
save_game(game_state)
load_game()
```

La logique métier délègue la persistance au module Données via ces interfaces abstraites.

4 Spécifications étendues

Références

[1] Christopher Obe. Carcassonne complete annotated rules. https://www.dover.nh.gov/Assets/government/city-operations/library/borrow/library-of-things/carcassonne-rules.pdf, 2014.

[2] Cathleen Heyden. Implementing a computer player for carcassonne. Master's thesis, Maastricht University, 2009. https ://project.dke.maastrichtuniversity.nl/games/files/msc/MasterThesisCarcassonne.pdf.